



Mastering Classification Fundamentals



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



A Map of Classification Methods

Tree-Based Methods

Decision Trees, Random Forests

Probabilistic Methods

Naïve Bayes

Distance-Based Methods

k-Nearest Neighbors (k-NN)

Linear Models

Linear and Logistic Regression model the direct relationship between features and class labels — simple, interpretable, and a strong baseline.

Margin-Based Methods

Support Vector Machines (SVM) go further by finding the optimal hyperplane that maximally separates classes in feature space.

Ensemble Methods

Bagging, Boosting, and Gradient Boosting push performance higher by combining multiple models into a single, more robust predictor.

Each family brings its own strengths — and the right choice depends on your data structure, problem type, and how much interpretability matters.

Training Dataset

To ground our classification concepts in practice, consider this real-world training dataset. The goal is to predict whether a customer **Buys Computer** — our target class — based on four key attributes: **Age**, **Income**, **Student**, and **Credit Rating**. With 14 labeled examples, this compact dataset captures a meaningful variety of customer profiles and purchasing behaviors.

Dataset at a Glance

- 14 training instances
- 4 predictor attributes
- 1 binary target variable
- Mix of categorical attributes

Class Distribution

- **Buys Computer = Yes:** 9 instances (64%)
- **Buys Computer = No:** 5 instances (36%)

Age	Income	Student	Credit Rating	Buys Computer
≤30	high	no	fair	no
≤30	high	no	excellent	no
31-40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31-40	low	yes	excellent	yes
≤30	medium	no	fair	no
≤30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤30	medium	yes	excellent	yes
31-40	medium	no	excellent	yes
31-40	high	yes	fair	yes
>40	medium	no	excellent	no

Reference: J. Han, J. Pei, and H. Tong, Data Mining: Concepts and Techniques, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.

A Shared Blueprint Across Every Example

Before diving into each classifier, recognize that **every example follows the exact same workflow**. Mastering this pattern is the central learning objective — once you see it, you'll apply it everywhere.

Load the data

Extract slevles (target labels).

Split the samples

Run `sample_random` and `train_test`.

Fit the model

Call the `fit` method on the estimator.

Predict (train)

Generate and evaluate predictions on the training set.

Predict (test)

Generate and evaluate predictions on the test set.

Code Structure

```
# 1. Load data and extract target levels
slevles <- levels(iris$Species)

# 2. Split samples
idx <- sample_random()
idx <- train_test(idx, nrow(iris))
iris_train <- iris[idx$train,]
iris_test <- iris[idx$test,]

# 3. Fit the model
model <- cla_??"Species", slevles)
model <- fit(model, iris_train)
```

```
# 4. Predict and evaluate on training set
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)

# 5. Predict and evaluate on test set
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
```

What Changes Between Examples

Only line 3 — where the model is created with `cla_??` — differs across examples. Everything else stays identical, making this pattern instantly recognizable and easy to reuse in any context.

As you study each classifier, pay attention not just to the constructor call, but to how the target is identified, how the split is constructed, and how training and test results should be interpreted independently.

Zero Rule – Establishing a Baseline

The **Zero Rule** (or *ZeroR*) is the simplest possible classifier: it **scans the training labels**, identifies the **most frequent class**, and always predicts that class at inference time — completely ignoring every input feature. It exists for a single purpose: to establish a **minimum performance floor**. Any meaningful model must surpass this threshold; if it cannot, it adds no real value over a trivial guess.

How It Works

- Scan all training labels and count class frequencies
- Identify the majority (most frequent) class
- At prediction time, always output that class — regardless of any input features

When It Matters

- Sets the performance floor every other model must beat
- If a sophisticated model fails to surpass it, that model adds no predictive value
- Useful for detecting severe **class imbalance** — a high ZeroR accuracy may signal that the dataset itself is skewed

Limitations

- Ignores all input features entirely
- Cannot generalize — it never adapts to new patterns
- Purely frequency-based: accuracy depends solely on class distribution in training data

Concrete Example — Computer Purchase Dataset

Training set: **No = 5, Yes = 9** (14 total cases)

Zero Rule identifies **"Yes"** as the majority class → always predicts **"Yes"**

Baseline accuracy: **9 / 14 ≈ 64.3%**

Any classifier on this dataset must achieve **> 64.3%** accuracy to be considered useful.

Baseline: cla_majority

```
# Loading the package
library(daltoolbox)

# Data
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
model <- cla_majority("Species", sleveles)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 0.358

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 0.233
```

Step-by-Step Walkthrough

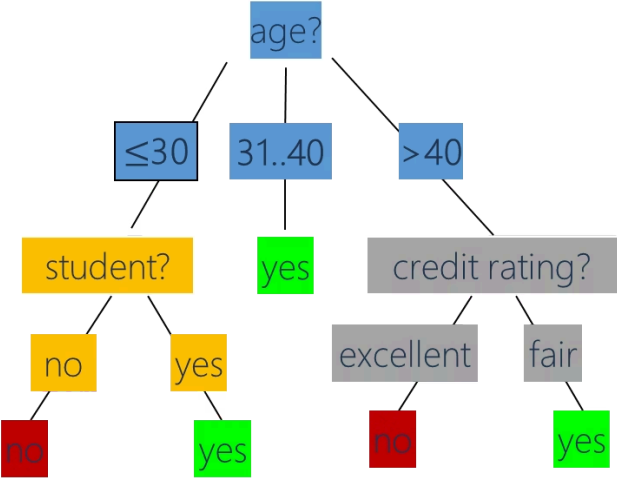
- Load the `daltoolbox` package and the `iris` dataset
- Extract the target variable levels using `levels()`
- Ensure reproducibility with `set.seed(1)`, then split the data with `sample_random()`
- Instantiate the model with `cla_majority()` and train it with `fit()`
- Evaluate the training and test sets independently with `evaluate()`
- Train accuracy: **0.358** | Test accuracy: **0.233** — your performance floor

📌 **Key Concept:** The majority classifier always predicts the most frequent class observed during training. It exists as a **minimum performance baseline** — every useful model you build must outperform it.

Building a Decision Tree

A **decision tree** is a hierarchical, flowchart-like model that **recursively partitions the feature space**. Each internal node tests an attribute, each branch represents an outcome, and each leaf node assigns a class label. The model is built **top-down** using a greedy divide-and-conquer strategy — always selecting the locally optimal split at each step.

Internal Nodes	Branches
Test a specific attribute's value — the node selects the attribute that best separates the data according to an impurity criterion such as information gain or Gini impurity	Each branch corresponds to a possible outcome of the parent node's test, routing each sample along the path that matches its attribute value toward a final decision
Leaf Nodes	
Assign the final class label — reached when a stopping criterion is met (pure node, maximum depth, or minimum sample count), representing the model's prediction	



Quinlan, J. R. (1993). **C4.5: Programs for Machine Learning**. Morgan Kaufmann. — The foundational reference for decision tree induction.

Key Characteristics

- Highly interpretable — the learned rules can be read directly from the tree structure
- No data normalization or feature scaling required
- Handles both numerical and categorical data natively
- Prone to overfitting on noisy data without pruning or depth constraints

Induction Strategy

- Greedy top-down recursive partitioning of the feature space
- Attribute selected by **information gain** (ID3 / C4.5) or **Gini impurity** (CART)
- Stopping criteria: pure node, maximum tree depth, or minimum samples per split

Decision Tree Induction Algorithm

Start at the Root

All training examples are placed at the root node, where the tree begins to take shape.

Recursively Partition the Data

From there, examples are split recursively based on selected attributes, branching deeper with each step.

Choose the Best Attribute

At each split, the most informative attribute is selected using heuristic or statistical measures such as information gain.

The Core Algorithm: A Greedy, Divide-and-Conquer Approach

The tree grows **top-down** through a recursive divide-and-conquer process. Attributes must be categorical — continuous values are discretized in advance. The algorithm stops partitioning a branch when one of three conditions is met:

Pure Node Reached

All samples at a given node belong to the same class — no further splitting is needed.

No Attributes Left

There are no remaining attributes to split on — the leaf is classified by majority vote.

No Samples Remain

There are no samples left to process at this branch of the tree.

A Quick Primer on Entropy

What Is Entropy – and Why Does It Matter?

Entropy measures the impurity or disorder within a dataset. The higher the entropy, the greater our uncertainty about which class any given example belongs to — making it a natural guide for where to split.

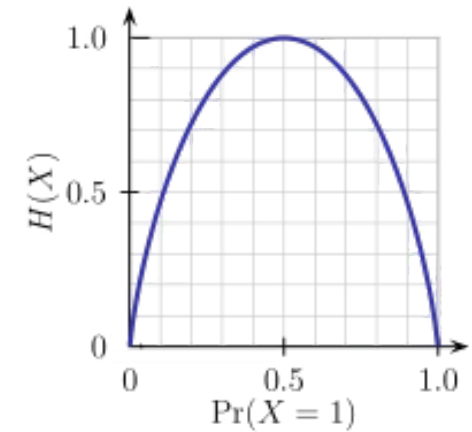
For a discrete random variable Y taking m distinct values $y_1, \dots, y_m$, entropy is defined as

$$H(Y) = \sum_{i=1}^m p_i \log(p_i)$$

where $H(Y)$ is the entropy and $p_i = P(Y = y_i)$. The formula weighs each class probability by its own logarithm, capturing exactly how much information is needed to describe the dataset — the more spread out the classes, the more information required.

Put simply: the higher the entropy, the greater the uncertainty.

$$H(Y|X) = \sum_{x \in X} p(x) H(Y|X = x)$$



Example with two classes (m = 2)

❏ **Practical Takeaway:** In binary classification, entropy peaks at 1.0 when classes are perfectly balanced (50%–50%) and drops to 0.0 when all examples belong to a single class — giving us a clear signal of how "mixed" a node really is.

Attribute Selection with Information Gain (ID3/C4.5)

Information gain is the core metric used by ID3 and C4.5 algorithms to determine which attribute most effectively splits the data at each node — the higher the gain, the better the split.

Measure the Reduction in Uncertainty

Information gain quantifies how much an attribute reduces entropy in the dataset. By comparing entropy before and after a split, we can rank attributes by how sharply they separate the classes.

Select the Most Informative Attribute

Building on that measurement, the algorithm evaluates every available attribute and selects the one with the highest gain — maximally reducing uncertainty about the target class at that step.

Grow the Tree Recursively

This process then repeats on each resulting subset, branching deeper until a stopping criterion is met — producing a tree built entirely from the most informative splits at every level.

Conceptual Formula: $Gain_X(Y) = H(Y) - H(Y|X)$

Attribute Selection: Information Gain in Action

The dataset below illustrates how information gain is calculated across different attributes to guide the splitting decision.

age	p_i	n_i	$H(p_i, n_i)$
≤ 30	2	3	0,971
31...40	4	0	0
> 40	3	2	0,971

Class P: buys computer = "yes" and Class N: buys computer = "no"

$$H(D) = 9/14 \log_2 (9/14) + 5/14 \log_2 (5/14) = -0.940$$

Evaluating the age attribute

$$H(D|age) = 5/14 I(2,3) + 4/14 I(4,0) + 5/14 I(3,2) = -0.694$$

$$Gain_{age}(D) = H(D|age) - H(D) = 0.246$$

Evaluating remaining attributes

$$Gain_{income}(D) = H(D|income) - H(D) = 0.029$$

$$Gain_{student}(D) = H(D|student) - H(D) = 0.151$$

$$Gain_{credit_rating}(D) = H(D|credit\ rating) - H(D) = 0.048$$

Compute Initial Entropy

Establish a baseline by measuring the impurity of the full dataset before any split occurs

Simulate Each Split

For every candidate attribute, compute the weighted entropy across the resulting subsets

Calculate the Gain

Subtract the weighted entropy from the initial entropy — the larger the difference, the more informative the attribute

Select the Best Attribute

Choose the attribute with the highest information gain to drive the split at this node

By repeating this process at every level of the tree, we consistently select the attribute that contributes most to correct classification — yielding models that are both efficient and interpretable.

Decision Tree: cla_dtree

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
model <- cla_dtree("Species", slevels)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 0.975

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 0.967
```

Step-by-Step Walkthrough

- Only one change from the baseline: swap in `cla_dtree()` as the model call
- The rest of the pipeline — split, fit, evaluate — stays exactly the same
- Train accuracy (**0.975**) slightly exceeds test (**0.967**), which is perfectly normal
- Avoid treating training accuracy as a reliable estimate of real-world performance
- Always extract target levels before fitting the classifier
- Only compare models trained on the same split for fair evaluation

📌 **Key Concept:** A decision tree recursively partitions the feature space to separate classes, producing a **highly interpretable model** — and an ideal starting point for understanding the full classification workflow.

The Same Tree, a Different Backend – `cla_rpart`

CART-style classification tree using the `rpart` backend, instead of the tree implementation behind `cla_dtree`.

```
model <- cla_rpart("Species")
model <- fit(model, iris_train)
prediction <- predict(model, iris_test)
evaluate(model, iris_test$Species,
         prediction)
```

- **Didactic goal:** keep the multiclass experiment fixed and change only the tree backend — a controlled comparison of wrapper behavior.
- **Same rhythm:** configure → fit → predict → evaluate, exactly like `cla_dtree`.
- **Talking point:** when two wrappers implement "the same" algorithm family, what differs is engine defaults and tuning knobs, not the experiment shape.

Reference: Breiman, L., Friedman, J., Olshen, R., and Stone, C. (1984). Classification and Regression Trees.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

